Faculty of Engineering
Ain Shams University

CSE281

Introduction to Artificial Intelligence

Item Price Prediction Competition

Submitted to

Dr. Mariam Nabil
TA. Heba
TA. Mohammad

Submitted by

Tarek Ahmed Abdelrahmen Hassouna 24P0313
Lujain Mohamed HelmiMohamed 24P0285
Ziad Elnoby Shehto Taea Ismail 24P0244
Marwan Ayman Sayed Draz 24P0246
Jana Ahmed Saieed Shaban 24P0410
Karem Adel Taha Ahmed 24P0325

10 may 2026

1. Objective

The goal of this project is to build regression models that predict the price-related target (Y) of retail items using a structured dataset containing both item-level and outlet-level features. The task involves data cleaning, feature engineering, model training, and submission of predictions on a held-out test set.

2. Dataset Overview

The dataset is a supervised regression problem based on anonymized retail data. It contains mixed numerical and categorical features.

Split	Rows	Columns	Target
Train	6,000	12	Y (item price)
Test	2,523	11	

Feature Summary

Column	Description	Type
X1	Item Identifier	Categorical
X2	Item Weight	Numerical
X3	Item Fat Content	Categorical
X4	Item Visibility	Numerical
X5	Item Type	Categorical
X6	Item MRP (Listed Price)	Numerical
X7	Outlet Identifier	Categorical
X8	Outlet Establishment Year	Numerical
X9	Outlet Size	Categorical
X10	Outlet Location Tier	Categorical
X11	Outlet Type	Categorical

3. Exploratory Data Analysis

Key Findings

- Target (Y) is approximately normally distributed, centered around 7–8 on a log scale. The distribution is left-skewed with most values between 6.5 and 8.5.
- Item MRP (X6) is the strongest predictor — there is a clear positive relationship with sales, with 4 distinct price bands visible in the scatter plot.
- Outlet Type has a significant impact: Supermarket Type3 has the highest average sales (~8), while Grocery Stores have the lowest (~5.5).
- Fat Content (Low Fat vs Regular) shows almost no difference in average sales (~7.3 each), making it a weak predictor.
- Item Type differences are minimal across all 16 categories (range ~7.0–7.4), suggesting limited predictive power after controlling for price.
- Location Tier shows a slight edge for Tier 2 outlets (~7.5) vs Tier 1 (~7.1).

- Outlet Age (derived from X8) shows no consistent trend — sales vary within each age group rather than across ages.

Missing Values & Preprocessing

Issue	Column	Fix Applied
Missing values	X2 (Item Weight)	Filled with median
Missing values	X9 (Outlet Size)	Filled with mode
Inconsistent labels	X3 (Fat Content)	Mapped: 'LF', 'low fat' → 'Low Fat'; 'reg' → 'Regular'
Zero visibility	X4 (Visibility)	Zero replaced with mean of non-zero values
Year to age	X8 (Year)	Converted to Outlet_Age = 2026 − X8
ID columns dropped	X1, X7	Dropped in some models (identifier-only)

4. Models Trained

Five different regression approaches were explored across the team:

Model	File	Validation MAE	Notes
Linear Regression	Linear_Model.py	0.421	Baseline; StandardScaler + OneHotEncoder
Ridge Regression	ridge.py	0.426	Alpha=1.0; minimal improvement over linear
XGBoost	XG_model_Sub_3.py	0.397	Early stopping, lr=0.01, max_depth=3
LightGBM	0.376.py	0.376	Categorical features, early stopping, Outlet_Age feature
CatBoost	0.373.py	0.373	Best model; MAE loss, depth=7, Outlet_Age & MRP features

4.1 Linear Regression

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import mean_absolute_error

df = pd.read_csv("train.csv")
df = df.drop(columns=["X1", "X7"])
df["X3"] = df["X3"].replace({
    "low fat": "Low Fat",
    "LF":      "Low Fat",
    "reg":     "Regular"
})

df["X2"] = df["X2"].fillna(df["X2"].median())
df["X9"] = df["X9"].fillna(df["X9"].mode()[0])

X = df.drop(columns = ["Y"])
y = df["Y"]

categorical_columns = X.select_dtypes(include=["object","string"]).columns.tolist()
numerical_columns = X.select_dtypes(exclude=["object","string"]).columns.tolist()

one_hot_encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_columns),
        ("cat", one_hot_encoder, categorical_columns)
    ]
)

X = preprocessor.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 50)

X_submission = pd.read_csv("test.csv")
X_submission = X_submission.drop(columns=["X1", "X7"])

X_submission["X3"] = X_submission["X3"].replace({
    "low fat": "Low Fat",
    "LF":      "Low Fat",
    "reg":     "Regular"
})

X_submission["X2"] = X_submission["X2"].fillna(X_submission["X2"].median())
X_submission["X9"] = X_submission["X9"].fillna(X_submission["X9"].mode()[0])
X_submission = preprocessor.transform(X_submission)
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

result = lin_reg.predict(X_test)
lin_mse = mean_absolute_error(y_test,result)
print(lin_mse)

result = lin_reg.predict(X_submission)
df = pd.read_csv("sample_submission.csv")
df["Y"] = result
df.to_csv("sample_submission.csv", index=False)
```

4.2 Ridge Regression:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge

df = pd.read_csv("train.csv")
df = df.drop(columns=["X1", "X7"])
df["X3"] = df["X3"].replace({
    "low fat": "Low Fat",
    "LF":      "Low Fat",
    "reg":     "Regular"
})

df["X2"] = df["X2"].fillna(df["X2"].median())
df["X9"] = df["X9"].fillna(df["X9"].mode()[0])
X = df.drop(columns = ["Y"])
y = df["Y"]

categorical_columns = X.select_dtypes(include=["object"]).columns.tolist()
numerical_columns = X.select_dtypes(exclude=["object"]).columns.tolist()

one_hot_encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_columns),
        ("cat", one_hot_encoder, categorical_columns)
    ]
)

X = preprocessor.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size = 0.3, random_state = 30)

ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train, y_train)
y_pred = ridge_model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
print(f"MAE: {mae:.4f}")
```

4.3 XGBoost:

```
mport pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
from xgboost import XGBRegressor

train = pd.read_csv("train.csv")
test  = pd.read_csv("test.csv")
sub   = pd.read_csv("sample_submission.csv")

train["X3"] = train["X3"].replace({"low fat": "Low Fat", "LF": "Low Fat", "reg": "Regular"})
train["X2"] = train["X2"].fillna(train["X2"].median())
train["X9"] = train["X9"].fillna(train["X9"].mode()[0])
train["X4"] = train["X4"].replace(0.0, train[train["X4"] > 0]["X4"].mean())
train = train.drop(columns=["X8"])

X = train.drop(columns=["Y"])
y = train["Y"]

cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
for col in cat_cols:
    X[col] = X[col].astype("category")

test["X3"] = test["X3"].replace({"low fat": "Low Fat", "LF": "Low Fat", "reg": "Regular"})
test["X2"] = test["X2"].fillna(train["X2"].median())
test["X9"] = test["X9"].fillna(train["X9"].mode()[0])
test["X4"] = test["X4"].replace(0.0, train[train["X4"] > 0]["X4"].mean())
test = test.drop(columns=["X8"])

cat_cols_test = test.select_dtypes(include=["object"]).columns.tolist()
for col in cat_cols_test:
    test[col] = pd.Categorical(test[col], categories=X[col].cat.categories)

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=20)

model_es = XGBRegressor( n_estimators=2000,learning_rate=0.01,
                         max_depth=3, subsample=0.85, colsample_bytree=0.8,
                         min_child_weight=5,gamma=0.1, random_state=1,
                         enable_categorical=True, tree_method="hist",
                         eval_metric="mae",early_stopping_rounds=150
)

model_es.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    verbose=200
)

best_n = model_es.best_iteration
val_mae = mean_absolute_error(y_val, model_es.predict(X_val))
print(f"\nBest n_estimators: {best_n}")
print(f"Validation MAE:    {val_mae:.4f}")

model_full = XGBRegressor(n_estimators=best_n,learning_rate=0.01,
                          max_depth=3, subsample=0.85,colsample_bytree=0.8,
                          min_child_weight=5,gamma=0.1,random_state=1,
                          enable_categorical=True,tree_method="hist",
                          eval_metric="mae",
)

model_full.fit(X, y, verbose=200)

preds = model_full.predict(test)
sub["Y"] = preds
sub.to_csv("XG_4_submission.csv", index=False)
```

4.4 LightGBM:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
import lightgbm as lgb
from lightgbm import LGBMRegressor

df = pd.read_csv("train.csv")
df["X3"] = df["X3"].replace({"low fat": "Low Fat", "LF": "Low Fat", "reg": "Regular"})
df["X2"] = df["X2"].fillna(df["X2"].median())
df["X9"] = df["X9"].fillna(df["X9"].mode()[0])
df["X4"] = df["X4"].replace(0.0, df[df["X4"] > 0]["X4"].mean())
df["Outlet_Age"] = 2026 - df["X8"]
df = df.drop(columns=["X8"])
X = df.drop(columns=["Y"])
y = df["Y"]

cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
for col in cat_cols:
    X[col] = X[col].astype("category")

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=20)

model = LGBMRegressor(objective="mae",n_estimators=5000,
                      learning_rate=0.02,num_leaves=10,
                      subsample=0.8,colsample_bytree=0.1,
                      min_child_samples=6,extra_trees=True,
                      reg_alpha=0.33,reg_lambda=0.37,
                      random_state=15,verbose=-1
)

model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    categorical_feature=cat_cols,
    callbacks=[lgb.early_stopping(200, verbose=False)]
)

preds = model.predict(X_val)
mae = mean_absolute_error(y_val, preds)
print(f"\nValidation MAE: {mae:.4f}")
print(f"Best iteration: {model.best_iteration_}")

test_raw = pd.read_csv("test.csv")
test_df = test_raw.copy()
test_df["X3"] = test_df["X3"].replace({"low fat": "Low Fat", "LF": "Low Fat", "reg": "Regular"})
test_df["X2"] = test_df["X2"].fillna(test_df["X2"].median())
test_df["X9"] = test_df["X9"].fillna(test_df["X9"].mode()[0])
test_df["X4"] = test_df["X4"].replace(0.0, test_df[test_df["X4"] > 0]["X4"].mean())
test_df["Outlet_Age"] = 2026 - test_df["X8"]
test_df = test_df.drop(columns=["X8"], errors="ignore")

for col in cat_cols:
    test_df[col] = test_df[col].astype("category")

test_preds = model.predict(test_df)

submission = pd.DataFrame({
    "row_id": range(len(test_preds)),
    "Y": test_preds
})
submission.to_csv("submission_lightgbm_3.csv", index=False)
print("submission_lightgbm.csv saved.")
```

4.5 CatBoost:

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
from catboost import CatBoostRegressor

train_raw = pd.read_csv("train.csv")
test_raw  = pd.read_csv("test.csv")

def preprocess(df):
    df = df.copy()
    df["X3"] = df["X3"].replace({"low fat": "Low Fat", "LF": "Low Fat", "reg": "Regular"})
    df["X2"] = df["X2"].fillna(df["X2"].median())
    df["X9"] = df["X9"].fillna(df["X9"].mode()[0])
    df["X4"] = df["X4"].replace(0.0, df[df["X4"] > 0]["X4"].mean())
    df["Outlet_Age"] = 2026 - df["X8"]
    df["Price_per_Visibility"] = df["X6"] / (df["X4"] + 0.001)
    df["MRP_x_Age"] = df["X6"] * df["Outlet_Age"]
    df = df.drop(columns=["X8"])
    return df

train = preprocess(train_raw)
test  = preprocess(test_raw)

X = train.drop(columns=["Y"])
y = train["Y"]
X_test = test.copy()

cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
cat_idx  = [X.columns.get_loc(c) for c in cat_cols]

def to_str(df):
    df = df.copy()
    for col in cat_cols:
        df[col] = df[col].astype(str)
    return df

X_str      = to_str(X)
X_test_str = to_str(X_test)

X_tr, X_val, y_tr, y_val = train_test_split(X_str, y, test_size=0.2, random_state=20)
cat_idx_tr = [X_tr.columns.get_loc(c) for c in cat_cols]

m_val = CatBoostRegressor(loss_function="MAE", iterations=3000,
                          learning_rate=0.05, depth=7,
                          l2_leaf_reg=1, random_seed=42, verbose=False
)
m_val.fit(X_tr, y_tr, cat_features=cat_idx_tr, eval_set=(X_val, y_val),
          early_stopping_rounds=100, use_best_model=True)

best_iter = m_val.best_iteration_
val_mae   = mean_absolute_error(y_val, m_val.predict(X_val))
print(f"Validation MAE: {val_mae:.4f}")
print(f"Best iteration: {best_iter}")

m_full = CatBoostRegressor(loss_function="MAE", iterations=best_iter,
                           learning_rate=0.05, depth=7,
                           l2_leaf_reg=1, random_seed=42,
                           verbose=False
)
m_full.fit(X_str, y, cat_features=cat_idx)

test_preds = m_full.predict(X_test_str)

submission = pd.DataFrame({
    test_raw.columns[0]: range(len(test_preds)),
    "Y": test_preds
})
submission.to_csv("submission_catboost_NOOOOTfinal.csv", index=False)
print("submission_catboost_final.csv saved.")
```

5. Best Model: CatBoost

The best submission was achieved using a CatBoost Regressor (0.373.py) with the following configuration:

Hyperparameter	Value
loss_function	MAE
iterations	Best from early stopping (up to 3,000)
learning_rate	0.05
depth	7
l2_leaf_reg	1
early_stopping_rounds	100

Feature Engineering

- Outlet_Age = 2026 − X8 (replaces raw establishment year)
- Price_per_Visibility = X6 / (X4 + 0.001) (interaction feature)
- MRP_x_Age = X6 × Outlet_Age (interaction feature)
- X4 zeros replaced with mean of positive values

Training Strategy

- 80/20 train-validation split (random_state=20) to find best iteration via early stopping.
- Retraining on full training data using the best iteration found above.
- Categorical columns passed natively to CatBoost (no encoding needed).

6. Results Summary

Model	Validation MAE
Linear Regression	0.421
Ridge Regression	0.426
XGBoost	0.397
LightGBM	0.376
CatBoost (Best)	0.373

Lower MAE is better. The CatBoost model with engineered features (Outlet_Age, interaction terms) achieved the best result at MAE = 0.373.

7. Visualization

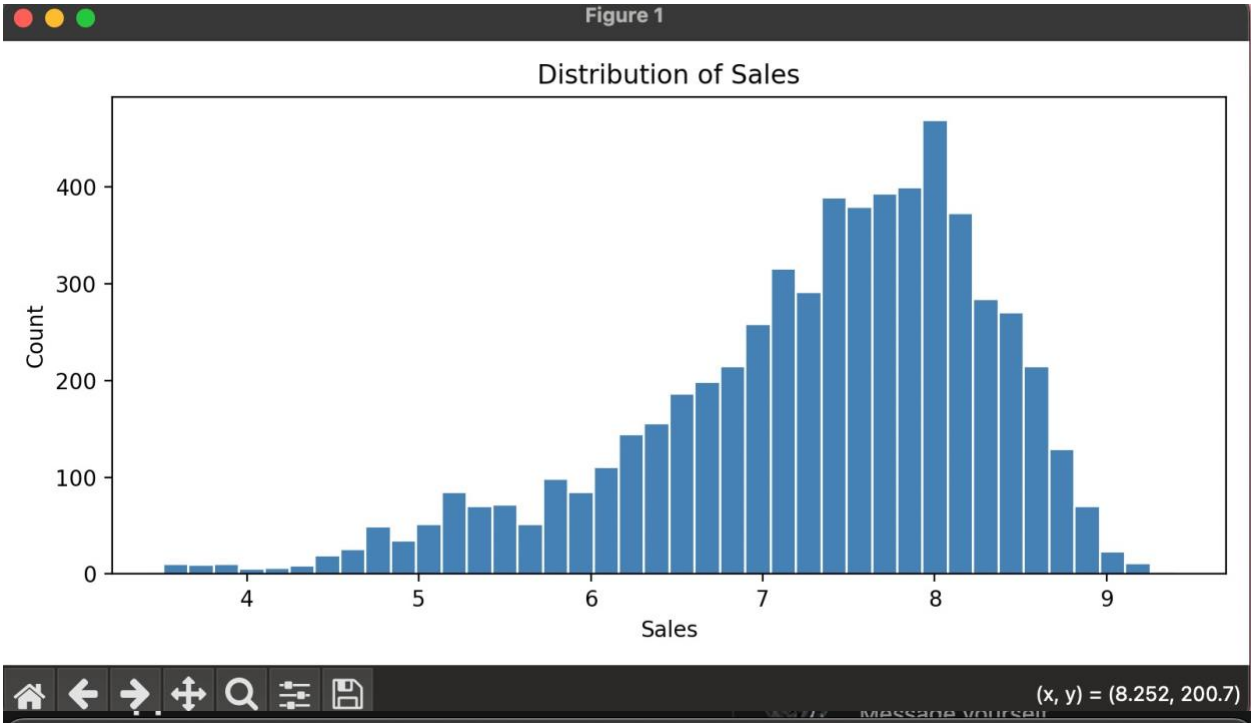
Figure 1
Distribution of Sales
Count
Sales
0
100
200
300
400
4
5
6
7
8
9
(x, y) = (8.252, 200.7)


Figure 1
Item Price vs Sales
Sales
Item MRP (Price)
4
5
6
7
8
9
50
100
150
200
250

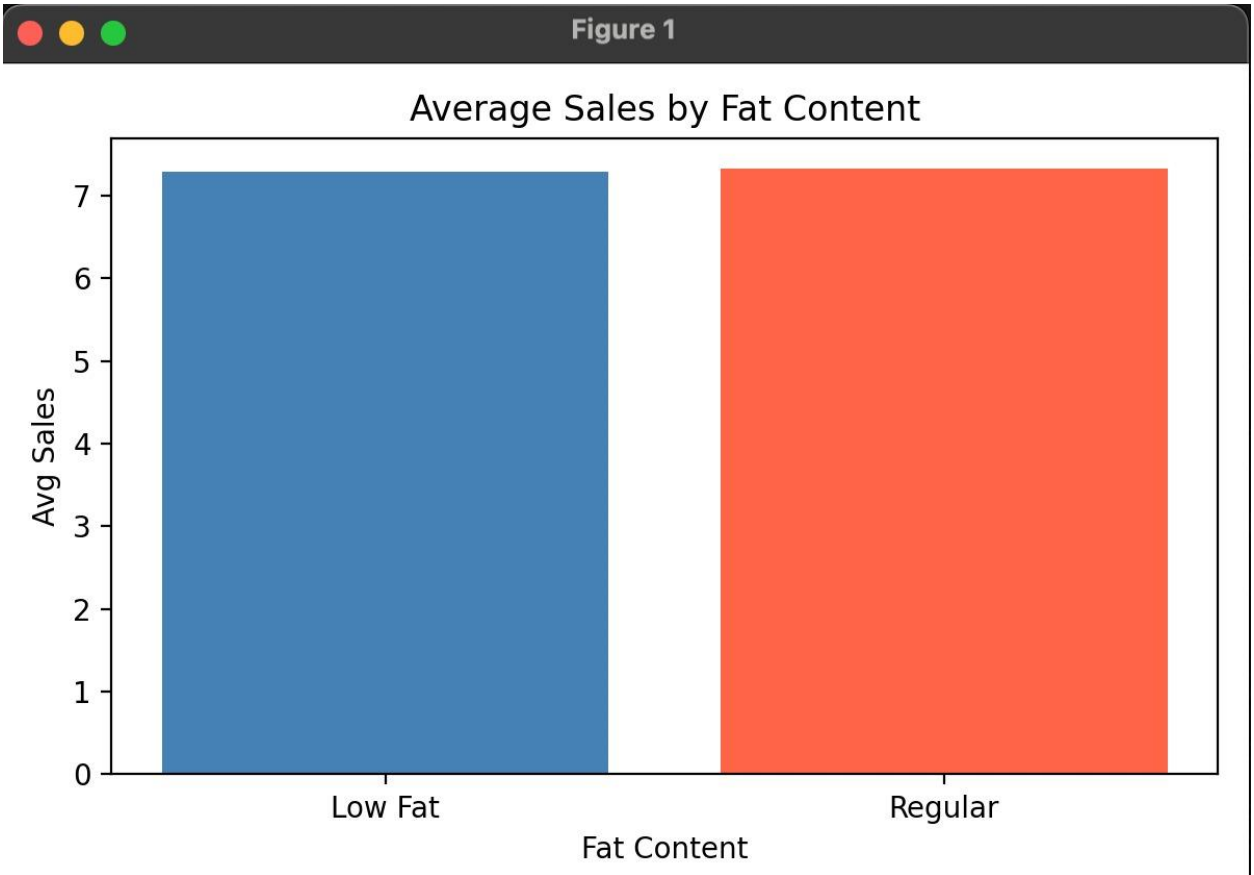
Figure 1
Average Sales by Fat Content
Avg Sales
0
1
2
3
4
5
6
7
Low Fat
Regular
Fat Content

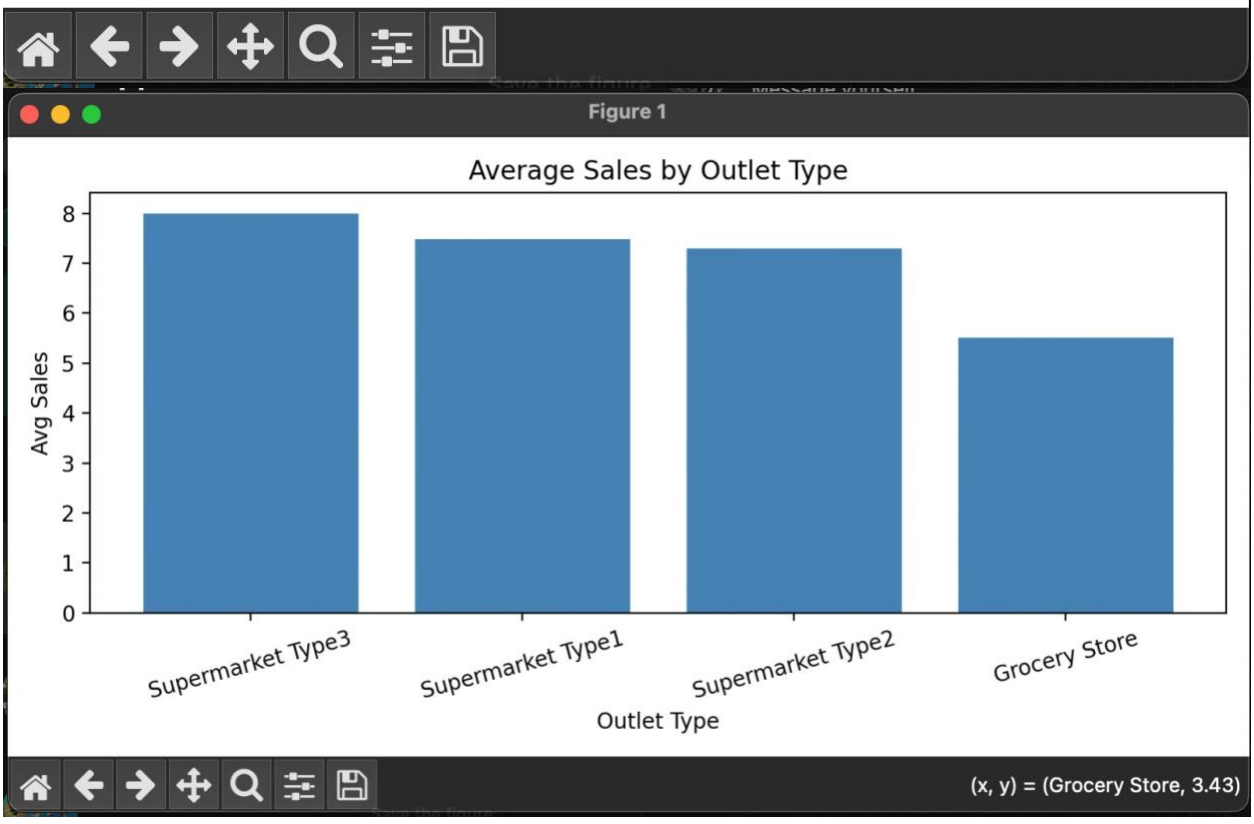
Figure 1
Average Sales by Outlet Type
Avg Sales
0
1
2
3
4
5
6
7
8
Supermarket Type3
Supermarket Type1
Supermarket Type2
Grocery Store
Outlet Type
(x, y) = (Grocery Store, 3.43)

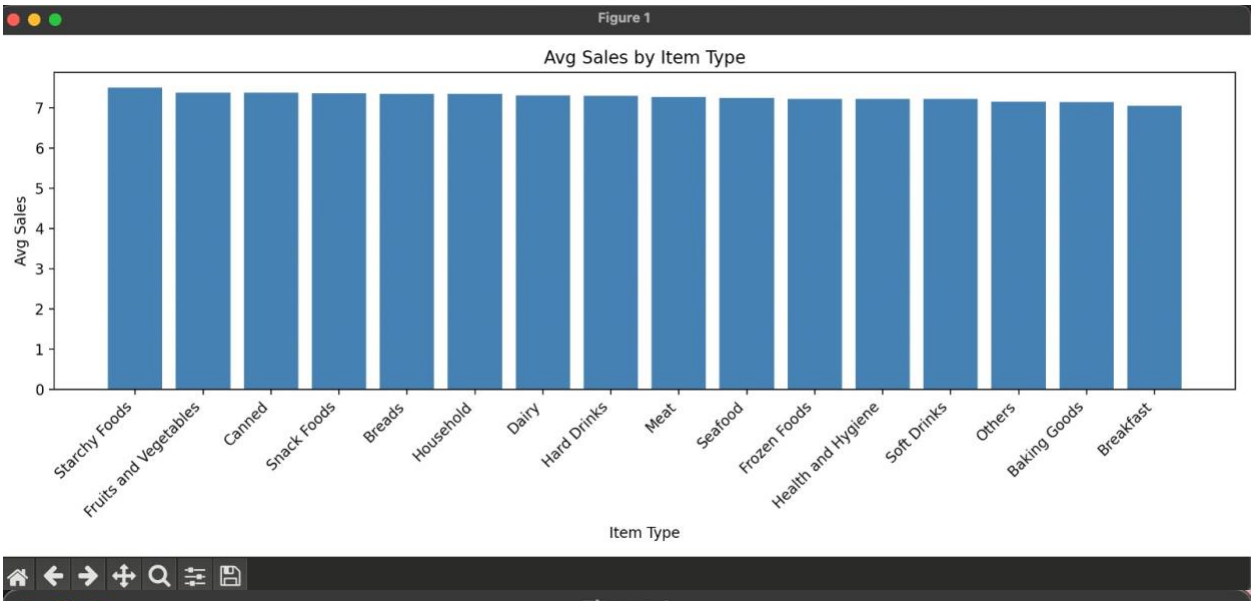
Figure 1
Avg Sales by Item Type
Avg Sales
0
1
2
3
4
5
6
7
Starchy Foods
Fruits and Vegetables
Canned
Snack Foods
Breads
Household
Dairy
Hard Drinks
Meat
Seafood
Frozen Foods
Health and Hygiene
Soft Drinks
Others
Baking Goods
Breakfast
Item Type

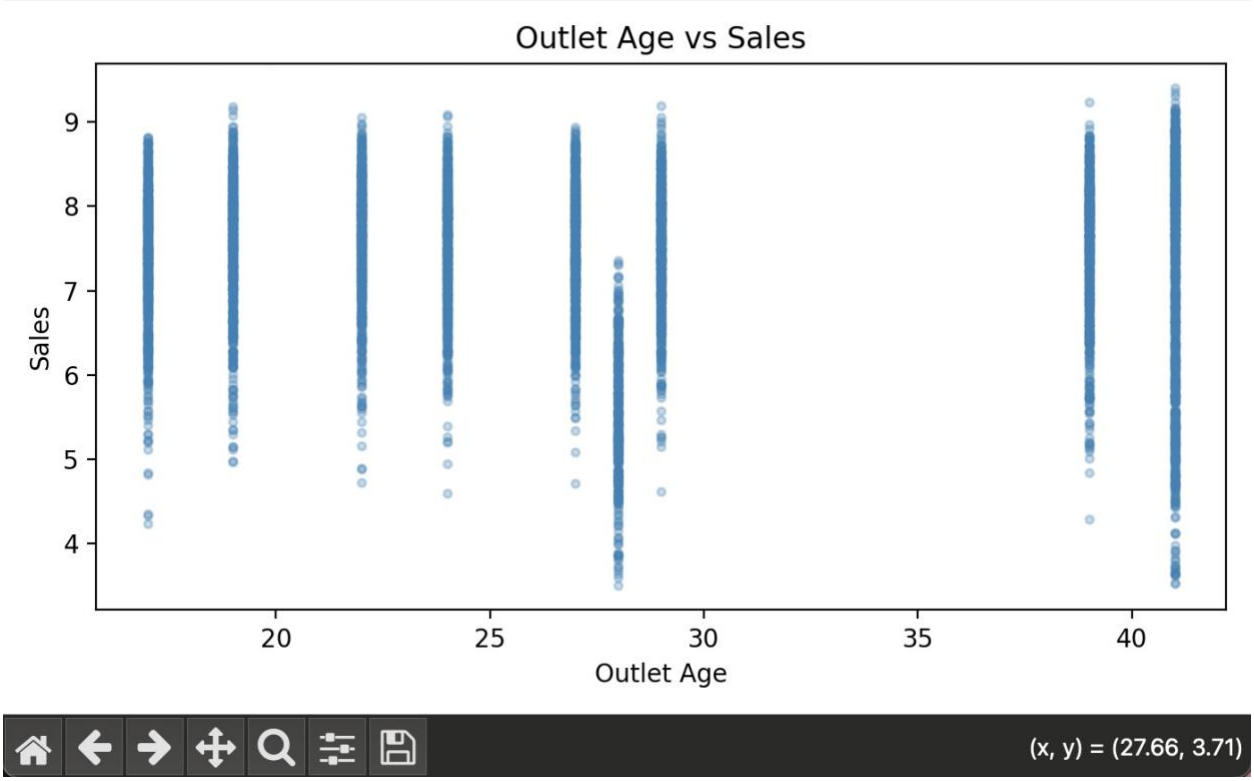
Figure 1
Outlet Age vs Sales
Sales
4
5
6
7
8
9
20
25
30
35
40
Outlet Age
(x, y) = (27.66, 3.71)

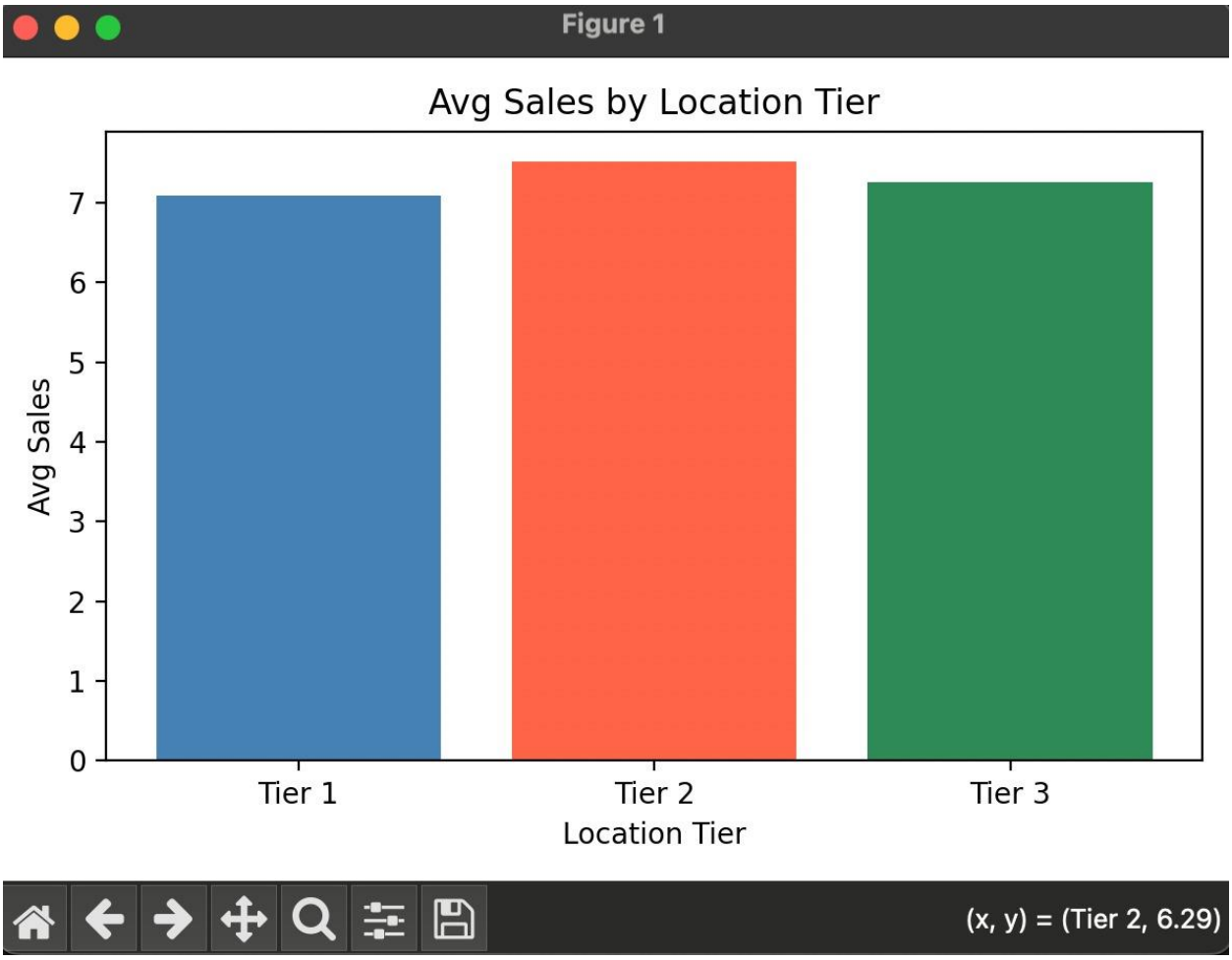
Figure 1
Avg Sales by Location Tier
Avg Sales
0
1
2
3
4
5
6
7
Tier 1
Tier 2
Tier 3
Location Tier
(x, y) = (Tier 2, 6.29)